

OWASP Secure Vault ve KVKK

Uyumluluk Portalı: Mimari, Hukuki ve

Çekirdek (Kernel) Seviyesi

Derinlemesine Analiz Raporu

Web tabanlı sistemlerin mimari güvenliği, uygulamanın çalıştırıldığı ortamın en alt katmanından başlayarak, veri işleme süreçlerinin yasal düzenlemelerle uyumluluğuna kadar uzanan bütünlük bir mühendislik disiplindir. "OWASP Secure Vault & KVKK Compliance Portal" isimli mimari proje, salt uygulama katmanı (Application Layer) savunmalarının ötesine geçerek; veritabanı sürücülerinin Soyut Sözdizimi Ağacı (AST) seviyesindeki davranışlarını, işletim sistemi çekirdeğinin (Linux Kernel) yetkilendirme algoritmalarını ve Kişisel Verilerin Korunması Kanunu (KVKK) ile 5651 Sayılı Kanun arasındaki hukuki gereksinimleri bir araya getiren derinlemesine bir güvenlik felsefesi üzerine inşa edilmiştir.

Bu rapor, söz konusu mimarinin arka planında yatan işletim sistemi düzeyindeki izolasyon mekanizmalarını, veritabanı derleyicisinin (compiler) bellekteki girdi işleme matematiğini ve uluslararası regülasyonların oluşturduğu hukuki paradoksları çözmek için kurgulanan adli bilişim (forensics) standartlarını ele almaktadır. Analiz süreci, yüzeysel tanımlamalardan tamamen arındırılarak, doğrudan sistem çekirdeği (kernel), bellek yönetimi (memory management), ağ protokolleri ve mahkeme/kurul içtihatları seviyesinde teknik ve hukuki bir derinlikle sunulmaktadır.

1. 50 Adımlık Çözümleme Felsefesi

Sistemin mimari, hukuki ve çekirdek katmanlarındaki güvenlik duruşunu bütünsel bir yaklaşımla analiz edebilmek için geliştirilen 50 adımlık çözümleme felsefesi, yapısal bir matris halinde aşağıda sunulmuştur. Bu adımlar, siber güvenlik savunma derinliğinin (Defense in Depth) her bir noktasını haritalandırmaktadır.

Faz	Adım	Disiplin	Kavramsal Derinlik ve Çözümleme Aksiyomu
Faz 1: Veritabanı AST ve Derleme Analizi	1	Mimari	Geleneksel metin katarı birleştirme (string concatenation) yöntemlerinin çalışma zamanındaki yapısal

			zafiyetleri ve komut enjeksiyonu riskleri tanımlanır.
	2	Mimari	SQLite VDBE (Virtual Database Engine) mimarisinde sqlite3_prepare_v2 C-API çağrısının SQL metnini nasıl bayt koduna (bytecode) derlediği haritalandırılır. ¹
	3	AST/Parser	Lemon ayrıştırıcı üreticinin (parse.y), Push-Down Automaton (PDA) mantığıyla bağlamdan bağımsız gramer (Context-Free Grammar) analizini yürütme şekli incelenir. ³
	4	AST/Parser	Girdilerin Soyut Sözdizimi Ağacı (AST) üzerindeki operasyonel düğümlere (nodes) ve yapraklara (leaves) dönüştürülme matematiği statik olarak modellenir.
	5	Bellek	sqlite3_bind_* fonksiyon ailesinin

			(örneğin sqlite3_bind_text), derlenmiş AST üzerinde yalnızca önceden tahsis edilmiş değişken adreslerine (memory pointers) nasıl referans verdiği kanıtlanır. ⁵
	6	Bellek	Sözcük analizörü (Lexer) aşamasının atlanmasının, zararlı yüklerin (payload) SQL komut zincirine (execution flow) dahil olmasını mimari olarak nasıl imkansız kıldığı açıklanır.
	7	Bellek	SQLITE_STATIC ve SQLITE_TRANSIENT bellek yönetim bayraklarının (flags), bellekteki verinin kalıcılığı, yığın (heap) tahsisi ve tampon bellek (buffer) üzerindeki etkileri ayrıştırılır. ⁶
	8	Operasyonel	randomblob(N) fonksiyonunun VDBE içerisindeki CPU döngü maliyetleri ve rastgele ikili veri (binary large object) tahsisinin I/O üzerindeki

			etkileri hesaplanır. ⁸
	9	Operasyonel	Zaman tabanlı (Time-based blind) SQL enjeksiyonlarında, sorgu çalışma zamanındaki anomalilerin (response-time anomaly) matematiksel dağılımı asenkron olarak ölçümlenir. ¹⁰
	10	Sentez	AST ve VDBE seviyesindeki statik derleme yapısı sayesinde, çalışma zamanında enjekte edilen zaman tabanlı saldırı yüklerinin neden yürütülebilir bir sözdizimi (sentaks) dalı oluşturamadığı sentezlenir.
Faz 2: Kriptografik İmha ve Veri Döngüsü	11	Adli Bilişim	Dosya sistemindeki standart silme işlemlerinin (örneğin işletim sistemi seviyesinde unlink), veritabanı "freelist" sayfalarında bıraktığı adli bilişim izleri (forensic artifacts) saptanır. ¹¹
	12	Veritabanı	SQLite özelinde PRAGMA secure_delete=ON

			komutunun derleme zamanı (compile-time) ve çalışma zamanı (run-time) davranışları, I/O gecikmeleriyle haritalandırılır. ¹¹
	13	Veritabanı	Veritabanı B-Tree sayfalarındaki tahsis edilmemiş (deallocated) blokların, disk sektörleri seviyesinde ardışık sıfırlarla (zeroing) üzerine yazılması mekanizması incelenir. ¹³
	14	Hukuk	KVKK Madde 7 (Unutulma Hakkı) kapsamında "Kişisel Verilerin Silinmesi, Yok Edilmesi veya Anonim Hale Getirilmesi" yükümlülüğünün teknik ve yasal sınırları net olarak çizilir. ¹⁵
	15	Hukuk	Mantıksal silme (Soft Delete) veya basit bayrak (flag) değişiminin, KVKK kapsamında neden "imha" olarak kabul edilemeyeceği Kurul kararları ışığında

			değerlendirilir. ¹⁶
	16	Adli Bilişim	Kriptografik silme ve veri üzerine yazma işleminin (Hard Delete), uluslararası adli bilişim (digital forensics) standartlarındaki veri kazıma (carving) tekniklerine karşı geri döndürülemezliği (irreversibility) test edilir.
	17	Veritabanı	Veritabanının SQLite WAL (Write-Ahead Logging) modunda çalışması durumunda, silinen verinin WAL dosyalarından da asenkron olarak tahliye edilmesi gereksinimi teknik olarak vurgulanır.
	18	Performans	Otomatik Vakumlama (Auto-vacuum) işleminin sayfa birleştirme (page defragmentation) sırasında yarattığı I/O maliyeti, secure_delete pragma statüsüyle

			karşılaştırılır. ¹¹
	19	Hukuk	Veri taşınabilirliği (Portability) yükümlülüğü (KVKK Madde 11, GDPR Madde 20) kapsamında kullanıcıya sunulan yapılandırılmış JSON çıktılarının şematik bütünlüğü denetlenir. ¹⁷
	20	Sentez	Sistemin veri yaşam döngüsünün, veri toplama fazından kriptografik imha fazına kadar kesintisiz ve matematiksel bir determinizm içerisinde güvence altına alınması sağlanır.
Faz 3: Hukuki ve Düzenleyici Çatışmalar	21	Paradoks	KVKK Madde 7'de tanımlanan kesin "imha" yükümlülüğü ile ISO 27001 (Ek A.8.15) loglama zorunluluğu arasındaki kavramsal ve hukuki çatışma (paradoks) izole edilir. ¹⁸
	22	Regülasyon	5651 Sayılı Kanun'un trafik kayıtları ve log yönetimi üzerine getirdiği 2 yıllık "değiştirilemez

			(WORM)" zaman damgalı saklama zorunluluğunun teknik mimariye etkisi incelenir. ²⁰
	23	Regülasyon	GDPR Madde 17 (Right to Erasure) kapsamında belirlenen "Meşru Menfaat (Legitimate Interest)" ve "Hukuki Yükümlülük (Legal Obligation)" istisnaları analiz edilir. ¹⁶
	24	Hukuk	Adli bilişim kapsamında tutulan bir audit.log içindeki "Kim, kimi, ne zaman sildi" bilgisinin salt bir kişisel veri işleme mi yoksa bir kanuni ispat (delil) aracı mı olduğu tartışılır. ²²
	25	İçtihat	2014 Google Spain (Mario Costeja González) emsal kararı referans alınarak unutulma hakkının mutlak bir hak (absolute right) olmaması prensibi değerlendirilir. ²²
	26	İçtihat	Kişisel Verileri Koruma Kurulu'nun (KVKK) veri ihlali bildirimleri (Örn:

			2020/927 ve 2020/935 sayılı kararlar) ve yetkisiz erişim loglarının tutulmaması üzerine verdiği cezai emsaller dengelenir. ²³
	27	Kriptografi	Log kayıtlarında takma adlandırma (Pseudonymization) kullanılarak, doğrudan kimlik tespitine yarayan verilerin (PII) kriptografik özetlerle (SHA-256 hash) temsil edilmesi kurgulanır. ²²
	28	Altyapı	Log verilerinin bütünlüğü için (WORM prensibi) blokzincir tabanlı veya elektronik sertifika hizmet sağlayıcısı (ESHS) tabanlı asimetrik zaman damgası mimarisi planlanır. ²⁰
	29	Mimari	Mahremiyet (Privacy) ile Hesap Verilebilirlik (Accountability) prensipleri arasında, veri minimizasyonu ilkeleri kullanılarak kriptografik bir köprü kurulur.

	30	Sentez	Regülasyonların "birbirini iptal etmediği, aksine üst üste yığıldığı (stacking)" gerçeği üzerinden, veri saklama ve imha işlemleri için hibrit bir uyumluluk stratejisi sentezlenir. ²⁵
Faz 4: Çekirdek (Kernel) İzolasyonu ve SUID	31	DevSecOps	Docker Compose yapılandırmasında deklare edilen security_opt: no-new-privileges:true parametresinin, User Space (Kullanıcı Alanı) katmanındaki görünürlüğü analiz edilir. ²⁶
	32	Kernel	prctl(PR_SET_NO_NEW_PRIVS, 1, 0, 0, 0) sistem çağrısının Linux Kernel (Çekirdek) API'si ile olan iletişimi haritalandırılır. ²⁸
	33	İşletim Sis.	Linux işletim sistemindeki Yetki Yükseltme (Privilege Escalation) vektörlerinin temel taşı olan SUID (Set Owner User ID) ve SGID bitlerinin dosya sistemindeki çalışma mantığı

			çözömlenir. ²⁸
	34	Kernel	execve(2) sistem çağrısının tetiklenmesi durumunda yeni bir süreç (process) yaratılırken Kernel'in fs/exec.c modölündeki check_unsafe_exec fonksiyonunun davranışı izlenir. ²⁹
	35	Kernel	task_no_new_privs(current) makrosunun dönüş değeriine göre Kernel'in bellekteki bprm->unsafe yapısına LSM_UNSAFE_NO_NEW_PRIVS bayrağını nasıl maskeleyerek eklediğı kanıtlanır. ²⁹
	36	LSM	Linux Güvenlik Modüllerinin (LSM - SELinux, AppArmor vb.), yeni süreçlerin ayrıcalıklarını belirlerken bu bayrağı nasıl bağlayıcı bir kısıtlama aracı olarak kullandığı incelenir. ²⁸
	37	Kernel	Yetki düşürme/koruma (Downgrade)

			operasyonunun kalbi olan security/commonca p.c kaynak dosyasındaki cap_bprm_creds_from_file fonksiyonu incelenir. ²⁹
	38	Matematik	Yetkilerin maskelenmesi (intersection) sırasında kullanılan $P'(permitted) =$ set matematiği formüle edilir ve kalıtsallık ispatlanır. ²⁹
	39	Saldırı Vec.	Olası bir RCE (Uzaktan Kod Çalıştırma) zafiyetinde, saldırganın konteyner içerisinde kök (root) erişimi sağlamak adına başvurduğu setuid ikililerinin (binaries) etkisiz kalma süreci modellenir. ²⁸
	40	Sentez	Konteyner kaçış (Container Escape) saldırılarına karşı, yetki devri matematiği kullanılarak çekirdek seviyesinde aşılmaz bir savunma

			derinliđi (Defense in Depth) sađlanır.
Faz 5: Entegrasyon, Test ve Yürütme	41	DevSecOps	Veritabanı AST güvenliđi ile Kernel no_new_privs güvenlik bariyerleri arasındaki ortogonal (birbirine dik ve bağımsız) savunma mimarisi entegre edilir.
	42	Sızma Testi	Zaman tabanlı (Time-based blind) SQLi testleri, randomblob ve sleep varyasyonlarıyla izole laboratuvar ortamında simüle edilerek bayt kod yalıtımı dođrulanır.
	43	Adli Bilişim	KVKK uyumlu Hard Delete işlemleri sonrasında hex editörler yardımıyla veritabanı ikili (binary) dosyalarında "Forensic Carving" (Adli Veri Kazıma) testleri yürütölür.
	44	Kriptografi	Zaman damgalı WORM log mekanizmasının bütönlüğü, kriptografik özetleme (SHA-256)

			ve asimetrik anahtar doğrulaması ile sınıanır.
	45	Sızma Testi	Konteyner içersisine kasıtlı olarak yerleştirilmiş SUID bitine sahip bir zararlı yazılım ile RCE üzerinden yetki yükseltme (Privilege Escalation) sızma testi (Pen-test) gerçekleştirilir.
	46	Kernel	Başarısız yetki yükseltme denemeleri Kernel logları (dmesg veya Auditd) üzerinden LSM red mesajları ile teyit edilir.
	47	Mimari	İzole mikroservis mimarisinde Flask uygulamasının belleğindeki context-aware yapıların (örn. Jinja2 autoescaping) Stored XSS direnci statik analiz araçlarıyla (SAST) onaylanır.
	48	CI/CD	Uygulama, Veritabanı ve İşletim Sistemi (Konteyner) katmanlarındaki DevSecOps sarmalı,

			sürekli entegrasyon (CI/CD) boru hatlarında zorunlu (blocking) standart haline getirilir.
	49	Uyumluluk	Regülatif denetimler (Kurul incelemesi, adli süreçler) anında sunulmak üzere Hukuki ve Teknik Uyumluluk ispat (evidence) paketleri otomatikleştirilir.
	50	Sonuç	Hukuki süreçlerde tam savunulabilirlik, teknik katmanlarda ise matematiksel imkansızlıklar üzerine inşa edilmiş sıfır güven (Zero Trust) mimarisi nihai olarak deklare edilir.

2. Araştırma Notları Şablonu

Aşağıdaki tablo, analiz boyunca incelenen kritik sistem bileşenlerini, regülasyonları ve çekirdek modüllerini indekslemektedir. Analiz boyunca argümanlar bu şablonun sunduğu referans düzlemi üzerinde inşa edilmiştir.

Kategori	İçerik ve Spesifik Parametreler
Araştırma Konusu / Proje	OWASP Secure Vault & KVKK Uyumluluk Portalı Derinlemesine Analizi
Kapsam	SQLite AST Mimarisi, Adli Loglama ile KVKK Paradoksu, Linux Kernel SUID İzolasyonu
Odaklanılan Zafiyetler	Blind SQL Injection (Time-based), Stored XSS, Privilege Escalation (Yetki Yükseltme),

	Adli Bilişim İzleri (Forensic Traces)
İncelenen Regülasyonlar	KVKK (Madde 7, Madde 11), GDPR (Madde 17, Madde 20), 5651 Sayılı Kanun, ISO 27001 (Ek A.8.15)
İncelenen Çekirdek (Kernel) Modülleri	fs/exec.c, security/commoncap.c, prctl, execve(2)
Veritabanı Bileşenleri	Lemon Parser (parse.y), VDBE (Virtual Database Engine), sqlite3_prepare_v2, sqlite3_bind_*
Kritik İşletim Sistemi İfadeleri	no-new-privileges:true, LSM_UNSAFE_NO_NEW_PRIVS, SUID/SGID
Kritik Mimari Kavramlar	secure_delete=ON, WORM, Zaman Damgası, Push-Down Automaton (PDA), Context-Aware Auto-Escaping

3. Teknik Odaklı Analiz: Prepared Statements Mimarisi, AST (Abstract Syntax Tree) Mantığı ve Zaman Tabanlı SQL Enjeksiyonlarına Karşı Bağışıklık

Geleneksel web mimarilerinde SQL Enjeksiyonu, genellikle metin katarı birleştirme (string concatenation) ve filtreleme atlatma hataları sonucunda veritabanının kontrol akışının manipüle edilmesiyle ortaya çıkar. Güvenlik literatüründe "SQLi veritabanına sızma" gibi yüzeysel tanımların aksine, bu zafiyetin temel kök nedeni, veri tabanı ayrıştırıcısının (parser) kodu ve veriyi aynı düzlemde değerlendirmesidir. OWASP Secure Vault projesinde, bu zafiyet string filtreleme (sanitization) gibi yüzeysel ve aşılabilir bir yöntemle değil, çekirdek veritabanı sürücüsü seviyesinde matematiksel ve yapısal bir izolasyon ile ("Prepared Statements" kullanarak) çözülmüştür. SQLite özelinde bu sürecin AST (Abstract Syntax Tree) mantığını, bayt kod üretimini ve randomblob gibi zaman tabanlı kör (time-based blind) enjeksiyonlara karşı bağışıklığını mimari düzeyde analiz etmek teknik ispat için elzemdir.

3.1. SQLite Sözdizimi Ayrıştırıcısı (Lemon Parser) ve Çalışma Zamanı Derlemesi (Compilation)

SQLite, SQL komutlarını derlemek ve VDBE (Virtual Database Engine) tarafından işlenebilecek

düşük seviyeli bayt kodlarına (bytecode) dönüştürmek için YACC/BISON alternatifi olan **Lemon Parser Generator** isimli bir ayrıştırıcı üretici kullanır.³ Lemon, parse.y dilbilgisi dosyası üzerinden bağlamdan bağımsız grameri (Context-Free Grammar) okuyarak bir Push-Down Automaton (İtmeli Otomat) üretir.³ Üretilen bu ayrıştırıcı, hatalara karşı son derece dirençli, thread-safe (iş parçacığı güvenli) ve yeniden girilebilir (reentrant) özelliktedir.³

Bir Python Flask uygulaması, SQLite veritabanına `sqlite3_prepare_v2()` C-API çağrısını gönderdiğinde, bu çağrı veritabanı motoru için bir "derleyici (compiler)" görevi görür.¹ SQLite mimarisinde derleme aşaması (Compilation) statik olarak üç ana fazda gerçekleşir:

1. **Tokenizer (Sözcük Analizörü):** Saf SQL sorgu metni (düz metin), SQL anahtar kelimeleri, operatörleri ve sabitlerini temsil eden mantıksal token'lara bölünür.
2. **Parser (Ayrıştırıcı):** Token'lar hiyerarşik bir yapı içerisinde değerlendirilerek bir AST (Soyut Sözdizimi Ağacı) oluşturulur.
3. **Code Generator (Kod Üretici):** AST üzerinde anlamsal (semantic) analiz yapılır (`select.c`, `build.c`, `expr.c`, `where.c` vb. modüller aracılığıyla). Bu aşamada AST üzerindeki döngüler, WHERE koşulları ve JOIN işlemleri VDBE sanal makinesinin çalıştırabileceği bayt koduna (bytecode) çevrilir.⁴

Aşağıdaki şema, metin tabanlı bir SQL komutunun AST üzerinden nasıl VDBE bayt koduna dönüştüğünü özetlemektedir:

[Flask / Python Uygulaması]

|

| (SQL Metni: "SELECT * FROM vault WHERE user_id =?")

v

[`sqlite3_prepare_v2()` C-API]

|

+++> 1. (Metni token'lara ayırır)

|

+++> 2. [Parser (`parse.y`)] (Push-Down Automaton ile AST inşa eder)

|

+++> 3. [Code Generator] (AST'yi anlamsal olarak analiz eder ve çevirir)

|

v

3.2. `sqlite3_bind` Ailesi, Parametre Bağlama ve Bellek Yönetimi

`sqlite3_prepare_v2()` kullanıldığında, SQL sorgusundaki kullanıcı girdilerinin yerini `?`, `?NNN`, `:VVV` veya `@VVV` gibi özel bağlama şablonları (binding templates) alır.⁵ Kod üretici (Code Generator), AST'yi inşa ederken bu şablonları birer operatör veya fonksiyon düğümü (function node) olarak değil, "tipi ve değeri henüz belli olmayan, hafıza adresinden yürütme zamanında çekilecek skaler bir değişken (literal node)" olarak ağaca ekler.

Sorgu AST'ye çevrilip derlendikten ve VDBE byte koduna dönüştürüldükten sonra, `sqlite3_bind_text()`, `sqlite3_bind_int()`, `sqlite3_bind_blob()` gibi fonksiyonlar çağrılarak bu parametrelere gerçek değerler atanır.⁵ Güvenliğin kilit noktası burasıdır: Veritabanı sürücüsü, parametre olarak gelen veriyi hiçbir şekilde yeniden ayrıştırıcıdan (lexer/parser) geçirmez.

sqlite3_bind_text(), yalnızca girdi verisinin bellekteki adresini (memory pointer) derlenmiş olan ifadeye (statement) skaler bir değer olarak bağlar.³⁶

Girdinin bellekteki kalıcılığı, C dilindeki makrolar olan SQLITE_STATIC veya SQLITE_TRANSIENT gibi bayraklarla belirlenir.⁶

- Eğer SQLITE_STATIC kullanılırsa, SQLite uygulamanın belleğindeki veriye doğrudan işaretçi (pointer) ile referans verir; bellek yönetimi uygulamaya aittir.
- Eğer SQLITE_TRANSIENT kullanılırsa, SQLite verinin bir kopyasını kendi özel bellek yığınınına (heap) alır.⁷ Her iki durumda da, veri asla çalıştırılabilir bir kod olarak değerlendirilmez; yalnızca karşılaştırma, atama veya taşıma operasyonlarında ham veri (raw data) olarak kullanılır.

3.3. Zaman Tabanlı (Time-Based Blind) SQL Enjeksiyonunun Matematiksel ve Mimari İmkansızlığı

Zaman tabanlı kör SQL enjeksiyonları, uygulamanın ekrana herhangi bir hata mesajı veya veri döndürmediği durumlarda, saldırganın veritabanına sorduğu evet/hayır sorularının cevabını almak için zaman anomalileri (response-time anomaly) yaratmasına dayanır. Geleneksel olarak bu tür saldırılarda veritabanında SLEEP() veya DELAY() fonksiyonları kullanılır.³⁷

Ancak SQLite veritabanında yerleşik bir SLEEP() fonksiyonu bulunmamaktadır. Bu nedenle saldırganlar, Ghost CMS Content API zafiyetinde (CVE-2026-26980) olduğu gibi, sistemi meşgul etmek için **CPU-bound (İşlemciye Bağımlı) Timing Oracle** adı verilen aşırı yük bindirici matematiksel ve kriptografik fonksiyon zincirleri kullanırlar.¹⁰ SQLite ortamında bu oracle genellikle aşağıdaki formülasyonla yaratılır:

$$E = \text{length}(\text{hex}(\text{randblob}(50000000)))$$

Bu ifade veritabanı ayrıştırıcısına gönderildiğinde sırasıyla şu operasyonel maliyetleri doğurur:

1. **randblob(50000000)**: İşletim sisteminden rastgele veri çekerek 50,000,000 baytlık (yaklaşık 50 MB) devasa bir sözde rastgele ikili büyük nesne (BLOB) üretir ve belleğe (heap) yazar.⁸
2. **hex(...)**: Bu fonksiyon, üretilen 50 MB'lık ikili blob'u onaltılık (hexadecimal) string formatına çevirir. Bu dönüşüm işlemi, verinin boyutunu 100,000,000 karaktere çıkarır ve veritabanı motorunu son derece yoğun bir bellek tahsisi (memory allocation) ve CPU dönüştürme

döngüsüne zorlar.⁸ Veri işleminin zaman karmaşıklığı $O(N)$ 'dir.

3. **length(...)**: Son olarak bu devasa dizinin karakter sayısı hesaplanır.¹⁰

Bir saldırgan, bu ağır operasyonu bir mantıksal AND veya OR operatörünün arkasına gizler. Eğer saldırganın tahmin ettiği veri doğruysa (TRUE branch), SQLite bu ağır yükü çalıştırmak zorunda

kalır ve sunucu HTTP yanıtını yaklaşık $400ms$ gibi tutarlı bir gecikmeyle döner.¹⁰ Yanlışsa

(FALSE branch), optimizasyonlar gereği arka blok çalıştırılmaz ve yanıt $10ms$ gibi çok kısa bir sürede döner.¹⁰ Saldırgan bu zaman farklarını statik analizle ölçerek veri tabanındaki her bir baytı sızdırabilir (data exfiltration).

Mimari Direncin Matematiksel Kanıtı:

Sistemin Prepared Statements mimarisi, bu tür bir CPU-bağılı komut dizisinin çalıştırılmasını mimari düzeyde engeller. Eğer sistemde güvensiz bir metin birleştirme (string interpolation) kullanılıyorsa, saldırgan girdisi olan:

```
' AND 1=LIKE('A', UPPER(HEX(RANDOMBLOB(50000000/2)))) --
```

metni, Lemon Parser'a doğrudan iletilecekti. Parser, AST ağacında OP_Function operasyon düğümünü tanımlayacak, randomblob ve hex fonksiyonlarını VDBE çalışma zamanı (run-time) için derleyecek ve böylece CPU yorulacaktı.

Ancak "OWASP Secure Vault" uygulamasında Flask, sqlite3_prepare_v2() kullanarak AST'yi yalnızca statik yapı üzerinden şu şekilde dondurmıştır:

```
SELECT * FROM users WHERE username =?
```

Daha sonra Flask uygulaması sqlite3_bind_text() çağrısı ile saldırganın gönderdiği o uzun 1' AND 1=LIKE('A', UPPER(HEX(RANDOMBLOB(50000000)))) metnini sisteme bir parametre olarak bağladığında şu gerçekleşir:

- Veritabanı motoru için atanan bu değer bir string V_{bind} değeridir.
- V_{bind} değeri, VDBE tarafından işlenecek yeni bir komut zinciri (Abstract Syntax Tree düğümleri kümesi) olarak değil, saf bir karakter dizisi (skaler string) olarak ele alınır.⁷
- VDBE, ağaçta yeni bir fonksiyon icrası (execution branch) yaratmaz; sadece veri tabanındaki username sütunu ile hafızadaki V_{bind} dizesini bayt-bayt (byte-to-byte) kıyaslar.
- randomblob, hex, ve LIKE gibi SQL operatörleri ile fonksiyon isimleri, parametrenin içinde yer alsalar dahi, ayrıştırıcı (parser) devreden çıktığı için fonksiyonel özelliklerini tamamen yitirmiştir.
- Çekirdek veya bellek açısından hiçbir CPU-bound operasyon veya devasa bellek tahsisi (memory allocation) gerçekleşmez.⁵

Bu mimari düzeydeki tip-güvenli bağlama (type-safe binding), mantıksal ağacın (AST) yapısının derleme zamanında (compile-time) dondurulmasını ve çalışma zamanında (run-time) enjekte edilen hiçbir zararlı girdinin, VDBE üzerinde yeni bir sentaks dalı (syntax branch) oluşturmamasını garanti ederek zafiyeti matematiksel olarak imkansız kılar.

4. Hukuki ve Adli Bilişim Paradoksu: KVKK Madde 7 (Unutulma Hakkı), ISO 27001 WORM Loglama Zorunluluğu ve 5651 Sayılı Kanun Çatışması

KVKK uyumluluk portalları tasarlanırken çözülmesi gereken en büyük ve karmaşık sistem mimarisi zorluklarından biri, veri sahibinin (Data Subject) "Unutulma Hakkı" (Right to be Forgotten) talebi ile sistem yöneticilerinin "Adli Bilişim (Forensics) ve Denetim İzi (Audit Trail)" zorunlulukları arasındaki yapısal ve hukuki çatışmadır (paradox). Bu durum, bir tarafın (Kanun) "Sistemi benden tamamen temizle ve iz bırakma" demesi, diğer tarafın (Diğer Kanun ve Standartlar) ise "Sistemde yapılan her işlemi kimin, ne zaman yaptığını mutlaka değiştirilemez şekilde loglamak zorundayım" demesiyle oluşur.¹⁹

4.1. KVKK Madde 7, Adli Veri Kazıma (Forensic Carving) ve Kriptografik İmha (Hard Delete)

KVKK Madde 7 ve "Kişisel Verilerin Silinmesi, Yok Edilmesi veya Anonim Hale Getirilmesi Hakkında Yönetmelik" gereği, işleme şartları ortadan kalkan kişisel veriler, ilgili kişinin talebi veya veri sorumlusunun re'sen inisiyatifi ile silinmek veya yok edilmek zorundadır.¹⁵ 2014 yılında görülen ünlü Google Spain (Mario Costeja González) davası, bu hakkın Avrupa genelinde GDPR Madde 17 altında "Unutulma Hakkı" olarak şekillenmesine ve regülasyonlara dahil edilmesine zemin hazırlamıştır.²²

Teknik mimaride, yazılım geliştiriciler genellikle DELETE FROM table WHERE id =? komutunu kullanarak veya "is_deleted = 1" gibi bayraklar (Soft Delete) ayarlayarak veri sildiklerini düşünürler. Ancak veritabanı mühendisliği ve adli bilişim (Digital Forensics) perspektifinden, standart bir DELETE komutu veriyi diskten fiziksel olarak silmez. SQLite mimarisinde, "auto-vacuum" mekanizması kapalı ise (ki performans için genellikle kapalıdır), silinen veriye ait B-Tree veritabanı sayfaları yalnızca "freelist" (boş liste) adı verilen tahsis edilmemiş bloklar yapısına eklenir.¹¹ Bu durumda veri, disk üzerinde fiziksel olarak okunabilir halde kalır ve herhangi bir "Forensic Carving" (Adli Veri Kazıma) aracı (örneğin Belkasoft) kullanılarak tamamen geri getirilebilir.¹²

Bu hukuki ihlali kökünden çözmek için, geliştirilen sistemde SQLite derleme zamanı veya çalışma zamanı seviyesinde PRAGMA secure_delete = ON konfigürasyonu aktifleştirilmiştir.¹¹ Bu pragma, silinen içeriklerin yer aldığı disk sektörlerinin (freelist sayfaları) üzerine ardışık sıfırlar (zeroing) yazılmasını zorunlu kılar.¹¹ "Zeroing" işlemi veritabanında I/O performansında belirli bir gecikmeye yol açsa da, verinin fiziksel veya manyetik kazıma yöntemleriyle bile kurtarılamayacağı anlamına gelir.¹¹ Bu teknik, veriyi yok ederek "Hard Delete" zorunluluğunu hukuken tam olarak karşılar.

4.2. ISO 27001 (Ek A.8.15) ve 5651 Sayılı Kanun Uyarınca Loglama Zorunluluğu

Kişisel veriler donanım seviyesinde sıfırlanarak geri döndürülemez şekilde yok edilmiş olsa da, Türkiye Cumhuriyeti sınırlarında veya global çapta hizmet veren sistemlerin uyması gereken diğer regülasyonlar devreye girmektedir. 5651 Sayılı "İnternet Ortamında Yapılan Yayınların Düzenlenmesi ve Bu Yayınlar Yoluyla İşlenen Suçlarla Mücadele Edilmesi Hakkında Kanun", internet erişimi sunan veya içerik barındıran sistemlerin, ağ trafik kayıtlarını (IP, MAC adresi, zaman aralıkları) 2 yıl boyunca saklamasını emreder.²¹ Bu kayıtların, kullanıcı müdahalesine kapalı (WORM - Write Once Read Many) bir şekilde ve "Zaman Damgası (Timestamp)" vurularak tutulması yasal bir zorunluluktur.²⁰ Sadece bir router veya basit metin belgesi ile log tutmak, yasal bir ispat niteliği taşımaz.⁴¹

Aynı zamanda, ISO 27001 Bilgi Güvenliği Yönetim Sistemi (Özellikle Ek A.8.15) standartları gereğince sistem yöneticisi eylemlerinin ve olay loglarının (audit logs) tutulması, olası bir siber ihlalde kök neden analizi (root cause analysis) yapılabilmesi ve adli sürecin aydınlatılması için şarttır.¹⁸

4.3. Hukuki Denge ve Sentez: Log Dosyası Bir Veri İhlali midir?

Paradoks Senaryosu: Bir kullanıcı sisteme başvurarak, "KVKK Madde 7 kapsamında bana ait tüm verileri sil (Hard Delete)" talebinde bulunur. İşlem gerçekleştirilir ve secure_delete=ON sayesinde veri tabanından kalıcı olarak (sıfırlanarak) silinir. Ancak, bu silme işlemini yetkili olarak gerçekleştiren sistem yöneticisinin eylemi, WORM yapısındaki harici bir audit.log dosyasına "Sistem Yöneticisi X, Kullanıcı Y'yi (Adı/Soyadı) Tarih Z'de sildi" şeklinde kaydedilir.²² KVKK açısından bakıldığında, "Kullanıcı Y'nin verisi aslında tamamen yok edilmemiştir; bir log dosyasına kopyalanmış ve orada tutulmaktadır. WORM mimarisi gereği log da silinememektedir. Bu durum kişisel veri ihlali midir, yoksa adli bilişim açısından yasal bir zorunluluk mudur?

Hukuki Analiz ve Regülasyon Dengesi: Regülasyonlar birbirini iptal etmez, aksine "üst üste yığılır" (stacking) ve kurum bu kesişimi doğru yönetmek zorundadır.²⁵ GDPR Madde 17 (Unutulma Hakkı) ve KVKK Madde 7, mutlak haklar değildir. GDPR Madde 17, veri işleme faaliyetinin, "hukuki bir yükümlülüğe uyulması" (compliance with a legal obligation) veya "yasal hakların tesisi, kullanılması veya savunulması" (establishment, exercise, or defense of legal claims) amacıyla gerekli olması durumunda silme hakkının reddedilebileceği istisnalarını barındırır.¹⁶ KVKK Madde 5/2-ç bendi "Veri sorumlusunun hukuki yükümlülüğünü yerine getirebilmesi için zorunlu olması" ve 5/2-e bendi "Bir hakkın tesisi, kullanılması veya korunması için veri işlemenin zorunlu olması" benzer istisnaları tanır.

Kişisel Verileri Koruma Kurulu (KVKK) Kararları incelendiğinde bu durum netlik kazanmaktadır. Kurul, veri ihlalleri yaşayan kurumlara "yetkisiz erişimlerin tespit edilememesi", "log kayıtlarının zaman damgalı tutulmaması", "Saldırı Tespit ve Önleme Sistemlerinden (IDS/IPS) log alınmaması" gibi nedenlerle ciddi idari para cezaları ("Veri Güvenliğinin Sağlanmaması") uygulamaktadır. Örneğin 08/12/2020 tarihli ve 2020/927 ile 2020/935 sayılı Kurul Kararlarında veri güvenliğine yönelik idari ve teknik tedbirlerin ihlali vurgulanmıştır.²³ Keza "Clickbus" veri sızıntısında, veri sızıntısının analiz edilebilmesi sistem loglarının incelenmesine bağlanmıştır.²³ Dolayısıyla, bir sistemin kendi güvenliğini, bütünlüğünü, hesap verilebilirliğini (Accountability) koruması ve siber suçlarda adli birimlere delil üretebilmesi (Meşru Menfaat - Legitimate Interest), veri sahibinin audit log'lardan açık metin olarak silinmesi talebinden hukuken üstün tutulabilir.¹⁷

Mimari Çözüm ve Tasarım: Loglama işlemi yasal bir zorunluluk (5651) ve meşru menfaat olsa dahi, "Veri Minimizasyonu" (Data Minimization) ilkesine harfiyen uyulmalıdır. audit.log dosyasına kullanıcının açık kimliği (PII) (Örn: "Ahmet Yılmaz silindi") düz metin olarak asla yazılmamalıdır.¹⁶ Bunun yerine, tasarlanan mimaride takma adlandırma (pseudonymization) ve tek yönlü özetleme (hashing) kullanılarak kriptografik bir denetim izi kurgulanmıştır.¹⁶ İşlem logu şu formattadır: ``

Aşağıdaki tablo, bu üç farklı silme yaklaşımının KVKK ve Loglama zorunluluğu açısından karşılaştırmasını sunmaktadır:

Yöntem	KVKK Madde 7 (Unutulma Hakkı)	5651 / Adli Loglama Uyumu	Kriptografik İmha (Forensic Carving)	Hukuki Risk

Soft Delete (is_deleted=1)	❌ İhlal (Veri hala B-Tree'de okunabilir)	✅ Uygun (Veri korunduğu için geçmişe dönük analiz mümkün)	❌ İhlal (Veri veritabanından disk sektöründe okunabilir durumda)	Yüksek Kurul Cezası
Hard Delete + Açık Metin Log (Plaintext Log)	❌ İhlal (Veri ana sistemden silinip loglara kopyalanmış, unutulma gerçekleşmemiş)	✅ Uygun (Tam adli iz ve zaman damgası)	✅ Uygun (secure_delete=ON devrede)	İkincil İhlal Riski
Hard Delete + Kriptografik Hashing Log	✅ Uygun (Ana veri yok edilmiş, logdaki veri geri çevrilemez özet)	✅ Uygun (Olayın kim tarafından gerçekleştirildiği zaman damgalı WORM olarak ispatlı)	✅ Uygun (secure_delete=ON devrede)	Uyumluluk Sağlandı

Bu mimari yaklaşım, WORM prensibine sahip ve 5651 Sayılı Kanun gereği elektronik sertifika hizmet sağlayıcısı (ESHS) tabanlı zaman damgası (timestamp) ile kilitlenmiş bir dosya sistemine eklendiğinde²⁰:

1. Adli bilişim (Forensics) açısından kimin hangi eylemi yaptığı kanıtlanabilir kalır; sistem yöneticisinin veya bir saldırganın "Ben silmedim" inkarı engellenir.
2. Açık kişisel veriler (PII) yer almadığı ve geri döndürülemez bir yapıda tutulduğu için KVKK Madde 7 ihlali oluşmaz.⁴⁰ Hem unutulma hakkı tesis edilir hem de yasal ispat yükümlülüğü matematiksel bir doğrulukla sağlanır.

5. DevSecOps ve Çekirdek Güvenliği:

no-new-privileges:true Parametresinin Linux Kernel Seviyesindeki İzolasyonu ve SUID Mimarisi

Modern mikroservis mimarilerinde web uygulamaları izolasyon amacıyla Docker gibi konteynerizasyon teknolojileri üzerinden yalıtılırlar. Ancak, Docker konteynerleri doğaları gereği tam tecrit edilmiş birer Sanal Makine (VM - Virtual Machine) değil, arka planda paylaşımlı bir

işletim sistemi çekirdeğinin (Linux Kernel) ad alanları (namespaces) ve cgroups kısıtlamalarıdır. Konteyner güvenliğinde en büyük risklerden biri, olası bir RCE (Remote Code Execution - Uzaktan Kod Çalıştırma) zafiyetinde saldırganın uygulamanın (örn: kısıtlı yetkilere sahip Flask/Python daemon) yetkileriyle sisteme sızdığında başvuracağı ilk saldırı aşaması olan **Yetki Yükseltme (Privilege Escalation)** durumudur.²⁸

Geliştirdiğiniz projede security_opt: no-new-privileges:true Docker Compose parametresi kullanılmıştır. Bu parametrenin yalnızca yüzeysel bir konfigürasyon (User Space) olmaktan çıkıp Kernel (Çekirdek) seviyesindeki belleğe nasıl intikal ettiğini ve çekirdek içerisindeki yetki maskelemelerini nasıl felç ettiğini analiz etmek, DevSecOps mimarisinin çekirdek felsefesini ortaya koyar.

5.1. SUID Bitleri, Yetki Yükseltme ve execve(2) Mantiğı

Linux dosya sisteminde bazı çalıştırılabilir dosyalar (executable binaries), örneğin /bin/su, /bin/sudo, veya /usr/bin/passwd, **SUID (Set Owner User ID)** veya **SGID (Set Group ID)** bitine sahiptir.²⁸ Normal şartlarda, sıradan kısıtlı bir kullanıcı (unprivileged user) bu programı çalıştırdığında, süreç (process) çalıştıran kullanıcının değil, dosyanın sahibinin (genellikle root kullanıcısının) yetkileriyle ayağa kalkar.³⁰

Bu mekanizma, Kernel seviyesinde execve(2) sistem çağrısının, başlatılan yeni programın yetkilerini (credentials), onu başlatan ebeveyn sürecin (parent process) yetkilerinden farklı ve daha yüksek bir seviyeye taşımasına izin vermesiyle gerçekleşir.²⁸ Saldırganlar RCE ile sisteme kısıtlı bir web kullanıcısı olarak girdikten sonra, sistemde bulunan bu SUID dosyalarını bularak kök (root) olmak isterler.²⁷

5.2. PR_SET_NO_NEW_PRIVS Bayrağının Kernel API Üzerinden Kalıtımı

Docker Compose dosyasındaki no-new-privileges:true (NNP) bildirimi, konteyner başlatılırken (container runtime/runc) Docker arka plan programının, konteynerin ana sürecini (PID 1) başlatmadan hemen önce Linux Kernel API'sine spesifik bir prctl() sistem çağrısı yapmasını tetikler²⁸:

```
C
PR_SET_NO_NEW_PRIVS 1 0 0 0
```

Bu kritik çağrı, ilgili sürecin (thread) çekirdekte tutulan özniteliklerine no_new_privs bayrağını ekler.⁴⁴ Linux 3.5 çekirdeğinden itibaren desteklenen bu mekanizmanın siber güvenlik açısından en önemli mimari özelliği şudur: Bir kere ayarlandığında, fork, clone ve execve çağrıları üzerinden tüm alt süreçlere (child processes) kalıtsal olarak aktarılır (inherited) ve bir daha Kernel API'si üzerinden **asla kaldırılamaz veya devre dışı bırakılamaz (cannot be unset)**.²⁸

5.3. Çekirdek (Kernel) Seviyesindeki Yetki Düşürme (Downgrade) İşleminin Anatomisi

Bir saldırgan, Flask uygulaması üzerinden bellek taşması (buffer overflow) veya deserialization gibi bir RCE bulduğunda ve konteyner içindeki bir SUID ikilisini (örneğin: `execve("/bin/su")`) çağırarak root yetkisine yükselmek istediğinde, Linux Çekirdeği C kaynak kodunda adım adım şu matematiksel operasyonlar yaşanır:

Adım 1: fs/exec.c İçerisinde Güvenlik Kontrolü (`check_unsafe_exec`) Saldırgan `execve` çağırmasını tetiklediğinde, Çekirdek yeni programı belleğe yüklemeye başlamadan önce `check_unsafe_exec` fonksiyonunu çalıştırır. Burada mevcut görevin (current task) NNP bayrağına sahip olup olmadığı `task_no_new_privs(current)` makrosu ile kontrol edilir ²⁹:

```
C
/* fs/exec.c dosyasından kesit */
static void check_unsafe_exec(struct linux_binprm *bprm)
{
    /* Bu mevcut güvenlik mekanizması (LSM) için katı kısıtlayıcıdır */
    if (task_no_new_privs(current) &&
        bprm->unsafe & LSM_UNSAFE_NO_NEW_PRIVS)
        return;
}
```

Eğer bayrak sistemde aktifse (ki Docker bunu set etmiştir), çekirdeğin ikili yürütme parametresi yapısındaki (`struct linux_binprm`) `unsafe` adlı bit-alanı değişkenine

`LSM_UNSAFE_NO_NEW_PRIVS` bayrağı mantıksal OR (`|=`) işlemi ile maskelenerek eklenir.²⁹

Adım 2: security/commoncap.c İçerisinde Yetki Kesişimi (`cap_bprm_creds_from_file`)

İşleyiş, Linux Güvenlik Modülleri (LSM - SELinux, AppArmor, Capabilities) aşamasına geldiğinde, yeni sürecin hangi yetkilerle çalışacağını belirleyen asıl kapı olan `cap_bprm_creds_from_file` fonksiyonu devreye girer. Bu aşama, başlatılacak dosyanın SUID veya SGID olup olmadığını `is_setid` değişkeni ile kontrol eder.²⁹


```

/* security/commoncap.c dosyasından kesit */
is_setid = is_setuid new old || is_setgid new old
if ((is_setid || cap_gained(permitted new old)) &&
    (bprm->unsafe & LSM_UNSAFE_PTRACE))

/* İhtiyaç dışı bir SUID çağırısı veya NNP bayrağı varsa downgrade başlat */
if (ins_capable new >user_ns, CAP_SETUID)
    (bprm->unsafe & LSM_UNSAFE_NO_NEW_PRIVS)

new -euid = new -uid
new -egid = new -gid
new -cap_permitted = cap_intersect new cap_permitted old->cap_permitted

```

Saldırganın kısıtlı bir kullanıcı alanında çağırdığı /bin/su bir is_setid hedefi olsa da (yani aslında root yetkisi vermek üzere yapılandırılmış olsa da), Kernel süreç yapısı üzerinden (bprm->unsafe & LSM_UNSAFE_NO_NEW_PRIVS) bitwise AND sorgusunun TRUE (doğru) döndüğünü tespit eder.²⁹ Bunun üzerine Kernel taviz vermeksizin bir "Yetki Düşürme / Reddi" (Downgrade) operasyonu başlatır:

1. **Kimlik İzolasyonu:** Yeni sürecin Etkin Kullanıcı Kimliği (new->euid), eski kısıtlı kullanıcı kimliğine (new->uid) eşitlenir.²⁹ Yani program sistemde SUID olarak yapılandırılmış dahi olsa Kernel, root etkinliğini ona vermez.
2. **Kapasite Kesintisi:** Hedef programın sahip olmak istediği yetkiler (new->cap_permitted), süreci başlatan mevcut kısıtlı uygulamanın (Flask uygulamasının) yetkileriyle kesişim kümesi (intersection - cap_intersect) işlemine tabi tutulur.²⁹ Kesişim matematiği

$$P'(permitted) = P(permitted) \cap old(permitted) \quad \text{formülünü uygular.}^{29}$$

Bu matematiksel modelleme, saldırganın mevcut kısıtlı kabuğu (shell) hangi sınırlı yetkilere sahipse, çalıştırdığı yeni SUID programının da ancak o kadar yetkiye sahip olmasını sağlar.²⁹ Yetkilerin alt süreçte genişlemesi matematiksel olarak engellenmiştir.

Aşağıdaki tablo, saldırganın (Flask Kullanıcısı) RCE üzerinden SUID programı çağırdığı durumlarda Linux Kernel'in no-new-privileges (NNP) bayrağına göre gösterdiği davranış farklılıklarını özetlemektedir:

Kernel Yetki Parametresi	NNP: FALSE (Varsayılan Docker/Linux)	NNP: TRUE (Uygulanan Mimari)
bprm->unsafe Durumu	Boş (Clear)	LSM_UNSAFE_NO_NEW_PRIVS atanmış

Etkin UID (euid)	Dosya sahibi UID (Genelde 0 / root)	Çağıranın Kısıtlı UID'si (Downgrade)
Elde Edilen Yetki Seti	Full Permitted Set (Tam root yetkisi)	Kısıtlı Intersection Kümesi ($P \cap old$)
Konteyner Kaçış Riski	Çok Yüksek (Host'a sığrama olası)	Sönümlendirilmiş (Kısıtlı alanda hapsoldü)
Saldırganın Nihai Erişimi	Tam Yetkili Shell (Root RCE)	Unprivileged (Yetkisiz) Kısıtlı Shell

Sonuç: RCE zafiyeti sistemde potansiyel olarak bulunmaya devam etse de (örneğin üçüncü parti bir kütüphanedeki bellek taşması zafiyeti), saldırganın mevcut sınırlandırılmış konteyner kullanıcılarından çıkıp (Privilege Escalation) Host (Ana Makine) seviyesine sığması (Container Escape) veya sistem üzerinde yetkili bir dosya modifikasyonu yapması Kernel'in donanım ve bellek izolasyonu seviyesinde reddedilir.²⁷ Kernel logları (dmesg veya Auditd) başarısız yetki yükseltme denemelerini LSM red mesajları olarak kaydeder, böylece savunma derinliği sağlanmış olur.

6. Genel Değerlendirme ve Sentetik Mimari Uyum

"OWASP Secure Vault & KVKK Compliance Portal" projesi, günümüzde sıkça rastlanan ve salt uygulama seviyesi filtrelemelere (Application-layer Sanitization) dayanan geleneksel "yamama" güvenlik anlayışını tamamen terk etmiş; bunun yerine çekirdek mimarisi (Core Architecture), bellek yönetimi (Memory Manipulation) ve uluslararası yasal düzenlemelerin doğasına inen bir (Defense-in-Depth) "Derinlemesine Savunma" stratejisi benimsemiştir.

Tasarımın her bir katmanı, bir diğer katmandaki olası bir çatlağı izole eden ortogonal (birbirine dik ve bağımsız) bariyerler oluşturmuştur:

- VDBE Bağlantı İzolasyonu:** Web uygulamalarında en kritik vektör olan SQL Enjeksiyonlarına karşı (özellikle analiz etmesi ve loglaması en zor olan zaman tabanlı "randomblob" gibi karmaşık vektörler), SQLite sürücüsünün VDBE bayt kod derlemesi (sqlite3_prepare_v2) ve statik bellek tahsisinin (sqlite3_bind) doğası gereği, ayrıştırıcının (parser) manipüle edilmesi ve işlemci çevrimlerinin (clock cycles) kötüye kullanılması matematiksel bir imkansızlık üzerine dondurulmuştur.
- KVKK / ISO 27001 Loglama Paradoksunun Çözümü:** Kişisel Verilerin kriptografik olarak disk sektörlerinden sıfırlanması (PRAGMA secure_delete=ON temelli Hard Delete) işlemi ile adli log tutma yükümlülüğü (5651 Sayılı kanun gereği Zaman damgalı WORM logları) birbirini dışlayan hukuki çelişkiler olarak değil; doğru bir kriptografik anonimleştirme/özetleme (SHA-256 Hashing) mimarisi ile birbirini tamamlayan Meşru Menfaat gereksinimleri (Legitimate Interest) olarak sistemin kalbine entegre edilmiştir. Bu

durum olası kurul incelemelerinde ve idari para cezası tehditlerinde kurumu mutlak suretle savunulabilir kılar.

3. **Kernel API İzolasyonu:** Docker yapılandırmasındaki no-new-privileges direktifi sayesinde, olası bir uygulama zafiyeti üzerinden elde edilen RCE durumlarında saldırganın yetki sınırları, işletim sistemi çekirdeğinin (Linux Kernel) C tabanlı kaynak kodlarındaki struct linux_binprm yapılarına eklenen LSM_UNSAFE_NO_NEW_PRIVS makrosu ve cap_intersect yetki kesişimi matematikleriyle mühürlenmiş; böylece saldırgan uygulama yetkilerine (User Space) hapsedilerek işletim sistemi (Kernel Space) tecridi sağlanmıştır.

Bu bütünsel mimari analiz, sürdürülebilir ve kurumsal düzeydeki bir sistem güvenliğinin yalnızca güvenli kod yazımı (secure coding) eylemine dayanmadığını; aksine, sistem teorisi (System Theory), işletim sistemi çekirdek davranışları (Kernel Behavior) ve yasal regülasyon matrislerinin uyumlu bir şekilde orkestre edilmesine dayandığını akademik, hukuki ve teknik bağlamda somut bir ispat olarak ortaya koymaktadır.

Alıntılanan çalışmalar

1. The Definitive Guide to SQLite, Second Edition (Expert's Voice in Open Source), erişim tarihi Haziran 2, 2026, <https://programmershouse.ir/Library/Files/SQLite.pdf>
2. Documentation - Fossil SCM, erişim tarihi Haziran 2, 2026, <https://fossil-scm.org/home/doc/a85cc7ce/src/sqlite3.c>
3. Architecture of SQLite, erişim tarihi Haziran 2, 2026, <https://sqlite.org/arch.html>
4. SQLite Internals: How The World's Most Used Database Works, erişim tarihi Haziran 2, 2026, <https://www.compileralchemy.com/books/sqlite-internals/>
5. Binding Values To Prepared Statements - SQLite, erişim tarihi Haziran 2, 2026, https://sqlite.org/c3ref/bind_blob.html
6. SQLite Requirements, erişim tarihi Haziran 2, 2026, <https://www2.sqlite.org/matrix/requirements.html>
7. How to bind parameters to LIKE with sqlite3_bind_text?, erişim tarihi Haziran 2, 2026, <https://sqlite-users.sqlite.narkive.com/GpLTDCLT/how-to-bind-parameters-to-like-with-sqlite3-bind-text>
8. PayloadsAllTheThings/SQL Injection/SQLite Injection.md at master - GitHub, erişim tarihi Haziran 2, 2026, <https://github.com/swisskyrepo/PayloadsAllTheThings/blob/master/SQL%20Injection/SQLite%20Injection.md>
9. Development of Automated Testing Scripts Using Robot Framework to Detect SQL Injection Vulnerabilities in APIs - IEEE Xplore, erişim tarihi Haziran 2, 2026, <https://ieeexplore.ieee.org/iel8/11392715/11393087/11393254.pdf>
10. Ghost CMS Content API Blind SQL Injection - CVE-2026-26980, erişim tarihi Haziran 2, 2026, <https://www.sonicwall.com/blog/ghost-cms-content-api-blind-sql-injection>
11. Compile-time Options - SQLite, erişim tarihi Haziran 2, 2026, <https://sqlite.org/compile.html>
12. Forensic data recovery with SQLite analysis in Belkasoft X, erişim tarihi Haziran 2, 2026, <https://belkasoft.com/sqlite>

13. Pragma statements supported by SQLite, erişim tarihi Haziran 2, 2026,
<https://sqlite.org/pragma.html>
14. SQLite deleted data stays in database binary file - Stack Overflow, erişim tarihi Haziran 2, 2026,
<https://stackoverflow.com/questions/42544858/sqlite-deleted-data-stays-in-data-base-binary-file>
15. T.C. Kültür ve Turizm Bakanlığı - KVKK - Kişisel Verileri Saklama ve İmha Politikası - TGA, erişim tarihi Haziran 2, 2026,
<https://tga.gov.tr/kvkk/kisisel-veri-saklama-ve-imha-politikasi>
16. GDPR Right to Erasure: What "The Right to Be Forgotten" Actually Requires, erişim tarihi Haziran 2, 2026,
<https://dev.to/custodiaadmin/gdpr-right-to-erasure-what-the-right-to-be-forgotten-actually-requires-493n>
17. Data Subjects: What They Are and Their Rights Under the GDPR - SearchInform, erişim tarihi Haziran 2, 2026,
<https://searchinform.com/articles/data-management/privacy/data-privacy/data-subjects/>
18. (PDF) Baseline Technical Measures for Data Privacy IN the Cloud (Updated), erişim tarihi Haziran 2, 2026,
https://www.researchgate.net/publication/381306612_Baseline_Technical_Measures_for_Data_Privacy_IN_the_Cloud_Updated
19. Special Delete Manager | Delete archive data GDPR-compliant - iTernity, erişim tarihi Haziran 2, 2026,
<https://iternity.com/en/products/special-delete-manager/>
20. Zaman damgası nedir? - 5651 Nedir, erişim tarihi Haziran 2, 2026,
<https://www.5651nedir.com/zaman-damgasi-nedir/>
21. 5651 Sayılı Kanun 5651 Sayılı Kanun Maddesinin Amacı 5651 Kanun maddesi kimleri kapsamaktadır? - DPU-WEB, erişim tarihi Haziran 2, 2026,
https://birimler.dpu.edu.tr/app/views/panel/ckfinder/userfiles/2/files/mevzuatlar/5651_Say_1_Kanun.pdf
22. Best Practices for GDPR-Compliant Data Deletion - Reform.app, erişim tarihi Haziran 2, 2026,
<https://www.reform.app/blog/best-practices-gdpr-compliant-data-deletion>
23. Yayımlanmış Kişisel Verileri Koruma Kurulu Kararları (2018 - 2021) - KVKK, erişim tarihi Haziran 2, 2026,
<https://www.kvkk.gov.tr/SharedFolderServer/CMSFiles/1d7c2f99-be2c-4971-a304-0a1eb3586bd1.pdf>
24. IT Offboarding Checklist for Security - Scribd, erişim tarihi Haziran 2, 2026,
<https://www.scribd.com/document/986423275/Employee-Offboarding-Checklist-Claud>
25. International Data Retention: Global Compliance Guide - OriginStamp, erişim tarihi Haziran 2, 2026,
<https://originstamp.com/en/blog/reader/international-retention-archiving-compliance-guide>
26. Scanning Docker infrastructure against CIS Benchmark with Wazuh, erişim tarihi Haziran 2, 2026,

- <https://wazuh.com/blog/scanning-docker-infrastructure-against-cis-benchmark/>
27. New Glibc Library Flaw Grants Root Access to Major Linux Distro - Cyber Kendra - Reddit, erişim tarihi Haziran 2, 2026, https://www.reddit.com/r/linux/comments/1aff4rp/new_glibc_library_flaw_grants_root_access_to/
 28. No New Privileges Flag — The Linux Kernel documentation, erişim tarihi Haziran 2, 2026, https://docs.kernel.org/userspace-api/no_new_privs.html
 29. Linux process capability change through execve syscall, erişim tarihi Haziran 2, 2026, <http://terenceli.github.io/%E6%8A%80%E6%9C%AF/2024/02/24/cap-change-execve>
 30. the setuid bit, or, how sudo minimally works - ops.tips, erişim tarihi Haziran 2, 2026, <https://ops.tips/notes/setuid-or-how-sudo-works/>
 31. security/commoncap.c - kernel/common - Git at Google - Android GoogleSource, erişim tarihi Haziran 2, 2026, <https://android.googlesource.com/kernel/common/+experimental-android-4.16/security/commoncap.c>
 32. erişim tarihi Haziran 2, 2026, https://uscope-linux.github.io/object_explorer/linux/security/commoncap.c.html
 33. How to execve a process, retaining capabilities in spite of missing filesystem-based capabilities? - Stack Overflow, erişim tarihi Haziran 2, 2026, <https://stackoverflow.com/questions/14636359/how-to-execve-a-process-retaining-capabilities-in-spite-of-missing-filesystem-b>
 34. turso/docs/manual.md at main - GitHub, erişim tarihi Haziran 2, 2026, <https://github.com/tursodatabase/turso/blob/main/docs/manual.md>
 35. Improve INSERT-per-second performance of SQLite - Stack Overflow, erişim tarihi Haziran 2, 2026, <https://stackoverflow.com/questions/1711631/improve-insert-per-second-performance-of-sqlite>
 36. SQLite Performance and Prepared Statements - Visual Studio Magazine, erişim tarihi Haziran 2, 2026, <https://visualstudiomagazine.com/articles/2014/03/01/sqlite-performance-and-prepared-statements.aspx>
 37. SQL Injection Payloads: How SQLi exploits work - Bright Security, erişim tarihi Haziran 2, 2026, <https://brightsec.com/blog/sql-injection-payloads/>
 38. sqlite3_example_bind.c - icculus.org, erişim tarihi Haziran 2, 2026, https://icculus.org/~chunky/stuff/sqlite3_example/sqlite3_example_bind.c
 39. KVKK - Fanset, erişim tarihi Haziran 2, 2026, <https://www.fanset.com/kvkk>
 40. Veri İşleme, Kaydetme, Paylaşma, Değiştirme, Anonimleştirme Ve İmha Politikası - Hakkari Üniversitesi, erişim tarihi Haziran 2, 2026, <https://hakkari.edu.tr/bilgiedinme/tr/sayfa/detay/4711/veri-isleme-kaydetme-paylasma-degistirme-anonimlestirme-ve-imha-politikasi>
 41. 5651 Sayılı Kanun ve Log Tutma Zorunluluğu - Yeni Nesil Bilişim – Dijital Dönüşümün Anahtarı, erişim tarihi Haziran 2, 2026, <https://yeninesilbilisim.com/5651-sayili-kanun-ve-log-tutma-zorunlulugu/>

42. Crowd: Right to erasure | GDPR Cloud - Atlassian Documentation, erişim tarihi Haziran 2, 2026,
<https://confluence.atlassian.com/spaces/GDPR/pages/949761091/Crowd+Right+to+erasure>
43. NO_NEW_PRIVS: avoiding privilege escalation - Marcos' Blog, erişim tarihi Haziran 2, 2026,
https://mpdesouza.com/blog/no_new_privs-avoiding-privilege-escalation/
44. PR_SET_NO_NEW_PRIVS(2const) - Linux manual page - man7.org, erişim tarihi Haziran 2, 2026,
https://man7.org/linux/man-pages/man2/PR_SET_NO_NEW_PRIVS.2const.html
45. design-proposals-archive/auth/no-new-privs.md at main - GitHub, erişim tarihi Haziran 2, 2026,
<https://github.com/kubernetes/design-proposals-archive/blob/main/auth/no-new-privs.md>